Turtle Graphics: An Easy Guide

Python's `turtle` module and the Oxford Turtle System, from your first line to recursive fractals

1. The big idea

Imagine a small robot turtle sitting on a sheet of paper with a pen attached to its tail. You give it simple orders such as walk 100 steps or turn left 90 degrees, and wherever it walks, it leaves a line.

At every moment the turtle only knows three things:

What	Meaning
Position	Where it is on the canvas, as (x, y).
Heading	Which direction it is facing, as an angle.
Pen	Whether the pen is down (drawing) or up (moving silently), plus its colour and thickness.

Every command changes one of these three things. If you can track position and heading in your head, you can predict any turtle drawing. That is exactly the skill quizzes test.

2. Your first program (Python)

Python ships with the turtle module, so there is nothing to install. Save this as `first.py` and run it:

```
import turtle

for i in range(4):
    turtle.forward(100)   # walk 100 steps
    turtle.left(90)       # turn left 90°

turtle.done()             # keep the window open
```



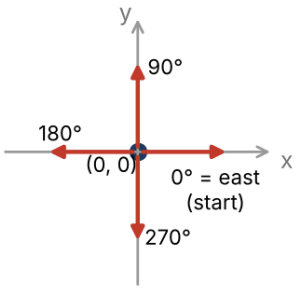
Result: a square

Tip: Write `from turtle import *` at the top and you can drop the `turtle.` prefix, so you just write `forward(100)`. Your code then looks almost exactly like Turtle System code.

Watch out: Never name your file `turtle.py`. Python would import your own file instead of the real module, and you'd get errors like `AttributeError: module turtle has no attribute forward`.

3. Coordinates and angles

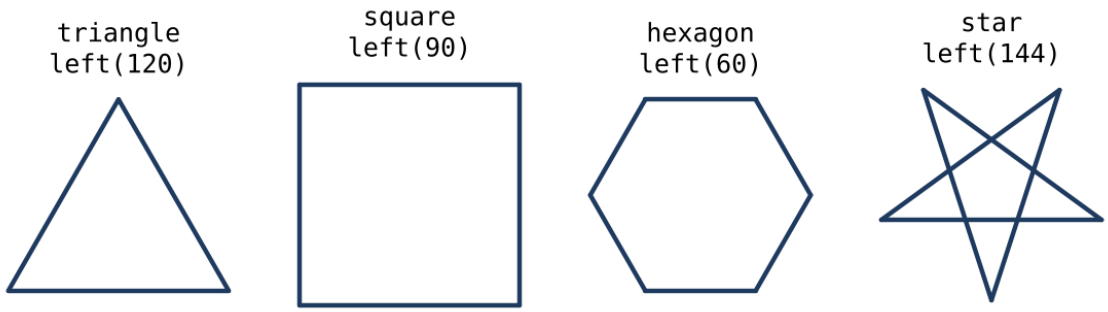
In Python's turtle:



- The turtle starts at the **centre**, `(0, 0)`.
- **x** grows to the right, **y** grows **upwards**.
- It starts facing **east** (heading 0°).
- `left` turns anticlockwise, `right` turns clockwise.
- `setheading(90)` points it north, whatever it was facing before.

The polygon rule. To draw a regular shape with n sides, repeat forward, turn n times and turn by **360 / n** each time. After all the turns the turtle has spun a full 360° and faces the way it started. Turning by 144° five times (720°, two full spins) makes a star.

```
def polygon(n, size):
    for i in range(n):
        turtle.forward(size)
        turtle.left(360 / n)
```



4. Command cheat sheet

The commands you'll actually need, with the Turtle System name where it differs. In Python, put `turtle.` in front, or use `from turtle import *`.

Python turtle	Turtle System	What it does
`forward(d)` / `fd(d)`	`forward(d)`	Walk d steps, drawing if the pen is down.
`back(d)` / `bk(d)`	`back(d)`	Walk backwards d steps. Heading doesn't change.
`left(a)` / `lt(a)`	`left(a)`	Turn anticlockwise by a degrees. Doesn't move.
`right(a)` / `rt(a)`	`right(a)`	Turn clockwise by a degrees. Doesn't move.
`penup()` / `pendown()`	`penup()` / `pendown()`	Lift or lower the pen (move without drawing).
`goto(x, y)`	`setxy(x, y)`	Jump to an absolute point.
`penup(); goto(...)`	`movexy(dx, dy)`	Move relative to where you are, without drawing (used in your code 1).
`setheading(a)`	`direction(a)`	Face an absolute direction.
`pensize(w)`	`thickness(w)`	Line thickness.
`color('red')`	`colour(red)`	Pen colour. Turtle System uses built-in colour names, no quotes.
`circle(r)`	`circle(r)`	Circle of radius r. **Careful:** Python draws it beside the turtle (centre to its left); Turtle System draws it around the turtle.
`dot(2*r, 'green')`	`blot(r)`	Filled circle. Python's `dot` takes the diameter, `blot` the radius.
`time.sleep(1)`	`pause(1000)`	Wait. Python uses seconds, Turtle System milliseconds.
`speed(0)`	(slider / options)	0 is the fastest in Python; 1 is slowest, 10 is fast.
`hideturtle()`	`hide`	Hide the turtle icon (faster, cleaner).
`done()`	not needed	Keeps the Python window open. Always the last line.

A few Turtle System names above (`setxy`, `direction`, `hide`) are from memory, not from your lab guide. If one doesn't work, check the QuickHelp tabs at the bottom of the Turtle System window.

5. Using the Oxford Turtle System

Your lab uses the Turtle System from turtle.ox.ac.uk: a single program (`TurtleSystem_15.exe`) that runs without installation. Its Python dialect is simplified, and the main differences from normal Python turtle are:

	Python turtle	Turtle System
Setup	`import turtle` needed	Nothing to import, commands are built in
Start heading	East (→)	**North (↑)**
Canvas	Centre is (0, 0), y grows up	1000 × 1000; (0, 0) is top-left and y grows **down**; turtle starts in the middle
End of program	`turtle.done()`	Not needed
Current state	`turtle.pos()`, `turtle.heading()`	`turtx`, `turty`, `turtd`, `turtt`, `turtc`

Running an example. Choose View → Simple Mode to hide advanced menus, open Examples, pick a program and press **RUN**. The guide recommends Examples 2 → Recursive factorials and Recursive triangles before the lab.

Watching recursion step by step. Your code 1 file puts `dump()` and `input(...)` inside the recursive functions. `input` stops the program until you press Enter, and `dump()` displays the program's current memory, so you can watch frames being added and removed. `pause(ms)` is the simpler option: it just slows the drawing down.

Tip: Because the two systems start facing different directions, the same code can come out rotated. In Python, add `turtle.left(90)` at the start to face north like the Turtle System.

6. The golden rule for drawing functions

When a function draws something, it should leave the turtle **exactly where it found it**: same position and same heading. This is what lets you call it again and again (or recursively) without everything drifting out of place.

Check the rule by adding up movement and turns:

• Polygon: n turns of 360/n = 360°, so the heading is unchanged, and the shape is closed, so the position is unchanged. ✓

• Tree branch: `forward(size)` … `back(size)`, and the turns cancel out (`left(30)`, `right(60)`, `left(30)`). ✓

Watch out: If a recursive drawing comes out lopsided or flies off the screen, the first thing to check is whether each call returns the turtle home.

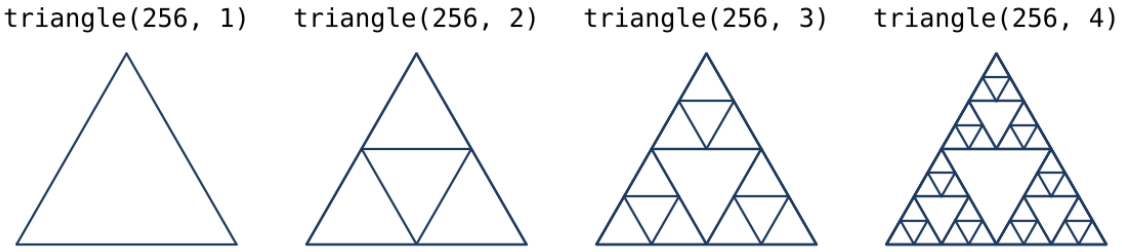
7. Recursive drawings

A recursive drawing is a function that draws a shape and, while drawing it, calls itself to draw smaller copies. Like every recursive function it needs a **base case** to stop. Use a `depth` counter (as in your code 3) or a minimum size such as `size > 1`.

Pattern A: shapes in the corners (Sierpiński triangle)

```
def triangle(size, depth):
    if depth > 0:                       # base case: depth 0 draws nothing
        for count in range(3):
            triangle(size / 2, depth - 1)   # small copy in this corner first
            turtle.forward(size)            # then one side of the big triangle
            turtle.left(120)

turtle.speed(0)
triangle(256, 3)
turtle.done()
```

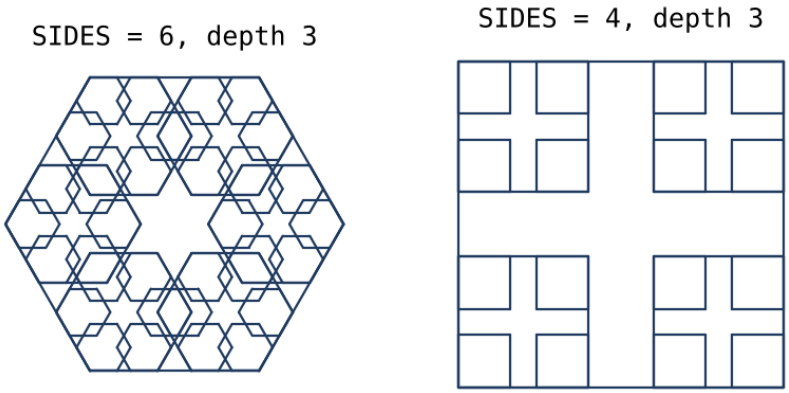


Each level adds three half-size copies. depth 1 is just a plain triangle.

Counting calls. Each call makes 3 more, so `triangle(256, 3)` runs 1 + 3 + 9 + 27 = **40** calls (the depth-0 calls count too), and at most 4 are on the stack at once.

Change the number of sides (and the turn to 360 / sides) to get other shapes:

```
def corners(size, depth, sides):
    if depth == 0:
        return
    for i in range(sides):
        corners(size * 0.4, depth - 1, sides)
        turtle.forward(size)
        turtle.left(360 / sides)
```

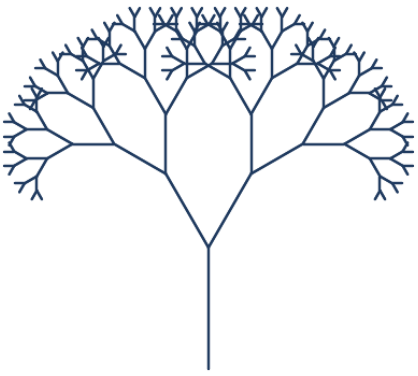


Pattern B: branching (trees)

Draw a trunk, turn, draw a smaller tree, turn the other way, draw another, then **undo** the turn and walk back down the trunk.

```
def tree(size, depth):
    if depth == 0:
        return
    turtle.forward(size)          # trunk
    turtle.left(30)
    tree(size * 0.7, depth - 1)   # left branch
    turtle.right(60)
    tree(size * 0.7, depth - 1)   # right branch
    turtle.left(30)               # face up again
    turtle.back(size)             # walk home

turtle.left(90)                   # face north
tree(100, 8)
```

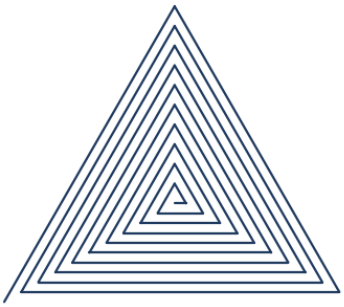


Pattern C: spirals

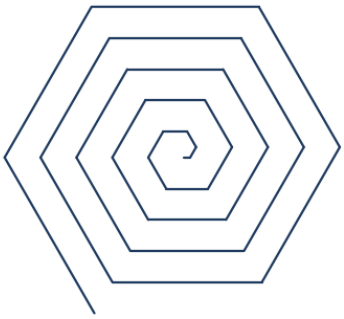
A spiral is the one case that does not return home: each call draws one segment, turns, and calls itself with a slightly longer length. The base case stops it when the length gets too big.

```
def spiral(size, step, angle, limit):
    if size > limit:                    # base case
        return
    turtle.forward(size)
    turtle.left(angle)
    spiral(size + step, step, angle, limit)
```

spiral(10, 10, 120, 300)



spiral(5, 5, 60, 150)



angle 120 makes a triangular spiral, angle 60 a hexagonal one

8. Common problems and fixes

Problem	Likely cause / fix
Window opens and closes instantly, or says Not responding	Add `turtle.done()` as the last line.
`AttributeError` for `forward`	Your file is called `turtle.py`. Rename it and delete any `turtle.pyc` / `__pycache__`.
`RecursionError: maximum recursion depth exceeded`	Base case never reached. `size / 2` never becomes exactly 0, so use `size > 1` or a depth counter.
Drawing is painfully slow	`turtle.speed(0)` and `turtle.hideturtle()`. For big fractals, `turtle.tracer(0)` at the start and `turtle.update()` at the end draws instantly.
Copies appear in the wrong places	A function doesn't return the turtle to its starting position/heading (section 6).
Figure is mirrored or rotated	`left` vs `right` swapped, or a different starting heading (Python east, Turtle System north).
`turtle.Terminator` error when re-running	Happens in some editors after closing the window. Just run again (or restart the editor).

9. Practice

Try these in order. Predict what you'll see before pressing run. That is the skill the quiz is testing.

1. Draw a hexagon with sides of 80 using a loop.

2. Write `polygon(n, size)` and draw a triangle, a square and an octagon next to each other (use `penup` / `pendown` to move between them).

3. In `triangle(size, depth)`, change `left(120)` to `right(120)`. What changes?

4. In the tree, change the angles to 45 / 90 / 45 and the ratio to 0.6. What happens if the turns don't cancel out (for example `right(50)` instead of `right(60)`)?

5. How many times is `tree` called for `tree(100, 3)`? (Each call with depth > 0 makes 2 more.)

6. Look at the sample lab figures. Which ones use pattern A, B or C?

Answer to 5: depths 3, 2, 1, 0 give 1 + 2 + 4 + 8 = **15** calls.